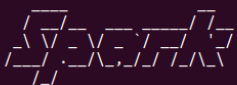


Write a simple streaming program in Spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc.), and print the cleaned text on the screen. (Open Ended Question).

Output:

```
scala> hadoop@bmscsecse-HP-Elite-Tower-600-G9-Desktop-PC:~$  
hadoop@bmscsecse-HP-Elite-Tower-600-G9-Desktop-PC:~$  
hadoop@bmscsecse-HP-Elite-Tower-600-G9-Desktop-PC:~$ spark-shell  
26/04/21 16:16:37 WARN Utils: Your hostname, bmscsecse-HP-Elite-Tower-600-G9-Desktop-PC resolves to a lo  
opback address: 127.0.1.1; using 10.124.3.70 instead (on interface eno1)  
26/04/21 16:16:37 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address  
Setting default log level to "WARN".  
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).  
26/04/21 16:16:39 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... usin  
g builtin-java classes where applicable  
26/04/21 16:16:40 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.  
Spark context Web UI available at http://10.124.3.70:4041  
Spark context available as 'sc' (master = local[*], app id = local-1776768400173).  
Spark session available as 'spark'.  
Welcome to  
 version 3.4.0  
  
Using Scala version 2.12.17 (OpenJDK 64-Bit Server VM, Java 11.0.23)  
Type in expressions to have them evaluated.  
Type :help for more information.  
  
scala> :paste  
// Entering paste mode (ctrl-D to finish)  
  
import org.apache.spark.sql.functions._  
import spark.implicits._
```

```
// Entering paste mode (ctrl-D to finish)  
  
import org.apache.spark.sql.functions._  
import spark.implicits._  
  
val lines = spark.readStream  
  .format("socket")  
  .option("host", "localhost")  
  .option("port", 9999)  
  .load()  
  .as[String]  
  
val stopWords = Seq("is", "the", "a", "an", "and", "to")  
  
val cleaned = lines.map { line =>  
  val lower = line.toLowerCase  
  val noSpace = lower.trim.replaceAll("\\s+", " ")  
  val noPunct = noSpace.replaceAll("[^a-zA-Z]", "")  
  
  val filtered = noPunct.split(" ")  
    .filter(word => !stopWords.contains(word))  
  
  val lemmatized = filtered.map {  
    case w if w.endsWith("ing") => w.dropRight(3)  
    case w if w.endsWith("ed") => w.dropRight(2)  
    case w => w  
  }  
  
  lemmatized.mkString(" ")  
}  
  
val query = cleaned.writeStream  
  .outputMode("append")  
  .format("console")  
  .start()  
  
query.awaitTermination()  
  
// Exiting paste mode, now interpreting.  
  
26/04/21 16:17:03 WARN TextSocketSourceProvider: The socket source should not be used for production ap  
plications! It does not support recovery.  
26/04/21 16:17:03 WARN ResolveWriteToStream: Temporary checkpoint location created which is deleted nor
```

```
// Exiting paste mode, now interpreting.

26/04/21 16:17:03 WARN TextSocketSourceProvider: The socket source should not be used for production ap
plications! It does not support recovery.
26/04/21 16:17:03 WARN ResolveWriteToStream: Temporary checkpoint location created which is deleted nor
mally when the query didn't fail: /tmp/temporary-39188d72-ef4d-4d55-a0c3-a405343ca7b2. If it's required
to delete it under any circumstances, please set spark.sql.streaming.forceDeleteTempCheckpointLocation
to true. Important to know deleting temp checkpoint folder is best effort.
26/04/21 16:17:03 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming D
ataFrames/Datasets and will be disabled.
-----
Batch: 0
-----
+-----+
|value|
+-----+
+-----+

-----
Batch: 1
-----
+-----+
|          value|
+-----+
|this runn test code|
+-----+

```

```
hadoop@bmscecse-H... x hadoop@bmscecse-H... x hadoop@bmscecse-H... x hadoop@bmscecse-H... x
hadoop@bmscecse-HP-Elite-Tower-600-G9-Desktop-PC:~$ nc -lk 9999
This is running and tested code!!!

```